

Performance Comparison of Async/Await vs Callback vs Streams in Node.js

Overview

This document analyzes the performance differences between using **async/await**, **callback-based I/O**, and **streams** in Node.js for writing a million lines to a file. It explores memory usage, CPU consumption, and execution time for each approach.

1. Why is the Callback API Faster than Async/Await?

Async/Await (Promise-based Approach)

- Uses `await` inside a loop, meaning **each iteration waits** for the previous write to finish.
- **Writes occur sequentially (one at a time)**, causing a **performance bottleneck**.
- The time complexity is **$O(N)$** for 1,000,000 writes, making it **very slow**.

Callback-based Approach

- Uses `fs.writeFileSync()`, which writes to the file **immediately**.
- Since there is **no artificial delay caused by `await`**, the operation runs **significantly faster**.
- The execution is **non-blocking** within the callback, but each synchronous write is still blocking.

Execution Time Comparison

Approach	Execution Time
Async/Await	413 seconds
Callback API	6.214 seconds

2. Memory Usage: Why Does Async/Await Use More Memory?

- `async/await` creates **1,000,000 promises**, leading to **higher memory consumption**.
- The event loop has to track promise resolutions, increasing memory overhead (**73.3 MB** used).
- Callback API avoids promises, requiring only **20 MB** of memory.

Memory Usage Comparison

Approach	Memory Usage
Async/Await	73.3 MB
Callback API	20 MB

3. CPU Usage: Why Does Async/Await Use More CPU?

- `async/await` introduces **context switching**, increasing CPU overhead (**25% CPU usage**).
- The callback approach does not introduce extra event loop overhead, keeping CPU usage lower (**20% CPU usage**).

CPU Usage Comparison

Approach	CPU Usage
Async/Await	25%
Callback API	20%

4. Optimized Approach Using Streams

Using **streams** provides the best balance between performance and memory efficiency. Streams write data in chunks, reducing memory overhead and improving speed.

```
const fs = require('fs');

const millionWritesOptimized = () => {
  console.time('writeFile');

  const writeStream = fs.createWriteStream('./text.txt');

  for (let index = 0; index < 1000000; index++) {
    writeStream.write(` ${index} `);
  }

  writeStream.end(() => {
    console.timeEnd('writeFile');
  });
};

millionWritesOptimized();
```

Advantages of Streams

- ✅ **Non-blocking:** Writes occur in chunks without blocking the main thread.
- ✅ **Lower Memory Consumption:** Prevents excessive memory usage by handling small buffers.
- ✅ **Faster Execution:** Performs better than async/await and callback-based approaches.

5. Final Performance Comparison

Approach	Memory Usage	Execution Time	CPU Usage
Async/Await	73.3 MB	413s	25%
Callback API	20 MB	6.214s	20%
Streams	Minimal	Very Fast	Low

6. Conclusion

- **Callback API is much faster** than `async/await` due to the lack of `await` bottlenecks.
- **Async/Await consumes more memory** due to promise handling.
- **Streams are the best choice**, as they provide optimal speed and memory usage.

For high-performance Node.js applications dealing with large file operations, **use streams** instead of `async/await` or callbacks for better efficiency.

 **Recommendation: Always prefer streams when handling large-scale file writes in Node.js for better performance and memory efficiency.**

Efficient File Handling in Node.js:

**`fs.promises.open()` VS
`fs.promises.writeFile()`**

Understanding `fs.promises.open()` in Node.js

When using the `fs.promises.open()` method in Node.js, it returns a **FileHandle** object, which provides an interface for interacting with the file in a more controlled and efficient manner compared to directly using `fs.promises` methods.

1. What is `fs.promises.open()` ?

The `fs.promises.open(path, flags[, mode])` function opens a file and returns a `Promise` that resolves to a `FileHandle` object.

```
const fs = require('fs').promises;

async function example() {
  try {
    const fileHandle = await fs.open('example.txt', 'r');
    console.log('File opened successfully');
    await fileHandle.close();
  } catch (err) {
    console.error('Error:', err);
  }
}

example();
```

- The `fileHandle` returned is an instance of the `FileHandle` class.
- It allows more efficient file operations compared to using `fs.promises` directly.

2. What is a `FileHandle` ?

A `FileHandle` represents an open file descriptor and provides methods to interact with the file. It ensures that the file remains open between multiple operations, reducing the need to repeatedly open and close the file.

Key properties of `FileHandle`

- `fileHandle.fd` : The file descriptor number.
- `fileHandle.read()` : Reads from the file.
- `fileHandle.write()` : Writes to the file.
- `fileHandle.close()` : Closes the file.
- `fileHandle.truncate()` : Truncates the file.

Example Usage

```
async function readFileWithFileHandle() {
  try {
    const fileHandle = await fs.open('example.txt', 'r');
    const buffer = Buffer.alloc(100);
    const { bytesRead } = await fileHandle.read(buffer, 0, 100, 0);
    console.log(`Read ${bytesRead} bytes:`, buffer.toString());
    await fileHandle.close();
  } catch (err) {
    console.error('Error:', err);
  }
}

readFileWithFileHandle();
```

- Here, `fileHandle.read()` allows us to read a specific number of bytes into a buffer.
- This is useful for partial reads without reading the whole file at once.

3. Difference Between `fs.promises.readFile()` and `fileHandle.read()`

Feature	<code>fs.promises.readFile()</code>	<code>fileHandle.read()</code>
File Handling	Opens and closes the file automatically	Requires manual opening and closing
Reading Method	Reads the entire file into memory at once	Reads a specific portion into a buffer
Performance	Not efficient for large files	More efficient for large files
Usage	Simple, one-line read operation	Requires handling a file handle manually

Example with `fs.promises.readFile()`

```
async function readFile() {
  try {
    const data = await fs.readFile('example.txt', 'utf-8');
    console.log(data);
  } catch (err) {
    console.error('Error:', err);
  }
}

readFile();
```

- `fs.readFile()` reads the entire file into memory at once.
- It is **not** efficient for large files because it loads the full content before processing.

Example with `fileHandle.read()`

```
async function readPartialFile() {
  try {
    const fileHandle = await fs.open('example.txt', 'r');
    const buffer = Buffer.alloc(64); // Read in chunks of 64 bytes
    let bytesRead;

    do {
      ({ bytesRead } = await fileHandle.read(buffer, 0, 64, null));
      console.log(buffer.toString('utf-8', 0, bytesRead));
    } while (bytesRead > 0);

    await fileHandle.close();
  } catch (err) {
    console.error('Error:', err);
  }
}

readPartialFile();
```

- Here, we read the file in chunks, making it **memory-efficient** for large files.
- We manually control how much data to read.

4. Efficient Writing to a File in a Loop:

`fs.promises.writeFile()` VS. `fileHandle.write()`

If you are writing millions of times to a file inside a loop, the best approach is crucial for efficiency.

Performance Bottleneck in `fs.promises.writeFile()`

```
const fs = require('fs').promises;

async function inefficientWrite() {
  for (let i = 0; i < 1_000_000; i++) {
    await fs.writeFile('output.txt', `Line ${i}\n`, { flag: 'a' });
  }
}

inefficientWrite();
```

Why is this inefficient?

- File Opens and Closes Repeatedly
- Slow I/O Performance
- More System Overhead

Optimized Approach Using `fileHandle.write()`

```
const fs = require('fs').promises;

async function efficientWrite() {
  const fileHandle = await fs.open('output.txt', 'w');

  for (let i = 0; i < 1_000_000; i++) {
    await fileHandle.write(`Line ${i}\n`);
  }

  await fileHandle.close();
}

efficientWrite();
```


Why is this more efficient?

- ✓ File remains open
- ✓ Avoids repeated file open/close
- ✓ Improves Performance
- ✓ Less system overhead

Buffered Writes for Even Better Performance

```
const fs = require('fs').promises;

async function bufferedWrite() {
  const fileHandle = await fs.open('output.txt', 'w');
  let buffer = '';

  for (let i = 0; i < 1_000_000; i++) {
    buffer += `Line ${i}\n`;

    if (i % 10_000 === 0) {
      await fileHandle.write(buffer);
      buffer = '';
    }
  }

  if (buffer.length > 0) {
    await fileHandle.write(buffer);
  }

  await fileHandle.close();
}

bufferedWrite();
```

Why Buffered Writes Are Even Better?

- ◆ Reduces I/O Operations
- ◆ Minimizes Disk Access
- ◆ Lower CPU & Memory Usage

5. Conclusion

Scenario	Recommended Method
Writing a small file occasionally	<code>fs.promises.writeFile()</code>
Writing continuously in a loop	<code>fileHandle.write()</code>
Writing millions of lines	Buffered <code>fileHandle.write()</code>

 **For best performance, use Buffered `fileHandle.write()` when writing large files!**